

PRESENCENET PROTOCOL SPECIFICATION (v0.1)

A Digital Memory Protocol for Sovereign Presence

Author: Mayowa Rosanwo

Steward: PresenceNet Foundation (Provisional)

Drafted: YYYY-MM-DD

Status: Genesis Specification (Draft)

License: CC BY-NC-SA 4.0

SECTION 1 — PURPOSE

PresenceNet is a digital protocol for verifiable presence. It exists to provide individuals with a sovereign, cryptographically verifiable way to declare that they were present in a specific moment. PresenceNet does not measure engagement, reputation, or value. It only records whether or not a person — identified by a digital handle — was present in a particular moment in time, optionally witnessed by others.

PresenceNet is not a social network. It is not a messaging layer. It is not a platform. It is a protocol — a layer for recording and anchoring the presence of people, not their performance.

This specification defines the core primitives, mechanics, and philosophical foundations of PresenceNet. It is intended to guide the construction of PresenceNet-compatible systems, applications, and extensions, while preserving the integrity and sovereignty of presence as a fundamental human signal.

SECTION 2 — CORE PHILOSOPHY

2.1 — Presence is Voluntary

Presence must never be coerced. The act of marking oneself present must originate from the individual. If presence is not voluntary, it is not presence — it is surveillance.

2.2 — Memory is Sacred

PresenceNet does not delete. Memory, once marked, is preserved. Trail entries may be hidden, but not erased. Presence is final.

2.3 — Identity is Emergent

PresenceNet does not rely on government IDs, KYC, or biometrics to assign identity. Identity emerges from the trail — the lived, marked history of an individual. If the trail is honest, the identity is strong.

2.4 — Performance is Noise

PresenceNet does not support likes, shares, feeds, or attention metrics. These are not signals of presence — they are distortions. PresenceNet supports quiet, sacred, immutable actions.

2.5 — Silence is Respected

Silence is not absence. In PresenceNet, choosing not to mark is itself an act of sovereignty. Silence must never be interpreted as disengagement, failure, or void. The system honors silence as deeply as it honors expression.

2.6 — The Trail is the Truth

The trail is the full record of valid marks a person has made and received. PresenceNet considers this trail as the canonical source of a person's digital existence. No further proof is needed.

2.7 — Coordination is Optional

PresenceNet enables coordination, but does not require it. People may coordinate around presence (e.g., in rituals or vaults), but presence itself is meaningful even when solitary.

2.8 — No Central Verification

PresenceNet is decentralized by design. There is no authority required to validate presence. If a mark is valid (signed, timestamped, recorded), it is presence.

2.9 — Presence is Not Surveillance

PresenceNet cannot be weaponized. It does not track, monitor, or expose. The system exists to protect human presence — not to exploit it. **All implementations must respect this boundary.**

SECTION 3 — GLOSSARY

Mark

→ A **Mark** is a signed, timestamped **signal of presence**, not a message. It is created when a user deliberately acknowledges a moment — either alone, toward another handle, or within a ritual. Marks are immutable, cryptographically signed, and form the atomic layer of PresenceNet.

Memo

→ A Memo is an optional, encrypted message attached to a Mark. It can contain anything the sender chooses — symbolic, personal, or procedural — and is never visible without decryption. Memos are not required, but when used, they are stored permanently and immutably with the Mark on the ledger or database.

Example: \$teresa includes an encrypted memo with her mark during a Memorial Ritual, storing a message only she and the recipient can later reveal.

Trail

→ An ordered collection of all valid Marks made and received by a handle. The Trail is a person's presence history and memory record. Trails are private by default but may include selectively public entries. Example: A user completing a 7-day Solitude Ritual will have 7 unique self-marks in their Trail.

Handle

→ A human-readable identity linked to a cryptographic wallet (e.g., [\\$elena](#)). Handles are used to send and receive Marks, initiate rituals, and anchor presence on the network. Each handle represents a unique trail, and a person may operate multiple handles for different roles or contexts.

Example: [\\$elena~public](#) may be used for open group rituals, while [\\$elena~family](#) is reserved for private, personal ceremonies.

Ritual

→ A structured, rule-based coordination pattern involving one or more participants. Rituals define the logic and mechanics of how presence is verified across time. They may include participation windows, trail conditions, biometric requirements, repetition rules, mutation logic, and more. Rituals may be solo, pairwise, or group-based. Upon completion, they can issue Completion Marks, unlock Chambers, or activate Vaults.

Example: A Circle Ritual invites 5 participants to mark within a 2-hour window to complete a shared memory.

Chamber

→ A sealed, presence-verified memory container created after the completion of an advanced ritual. Chambers hold optional symbolic content (e.g., text, links, unlocks) and display only the Marks that participants opt to make public. Example: Completing a seasonal reflection ritual may unlock a Chamber with a letter from the host and an invitation to the next ritual tier.

Vault

→ A multi-user, multi-signature XRPL wallet controlled by presence-based rules. Vaults

are used for financial coordination or access control, and must meet configured signing thresholds. *Example: A Vault may require 3 of 5 Circle Elders to mark a ritual before releasing stored XRP.*

Witness

→ An optional third-party handle that confirms the presence of another by co-signing or receiving a Mark. Witnesses add social or ceremonial verification. *Example: A Time Capsule may require a Witness Mark from \$guardian to become valid.*

Completion Mark

→ A special Mark issued by the system or ritual host after a participant has fulfilled all ritual conditions. It anchors the completion event in the Trail and may unlock future access. *Example: A Completion Mark may state “Circle Ritual completed: Jan 1, 2026.”*

Presence Layer

→ The dynamic sum of all active Marks, Rituals, Trails, Chambers, and Vaults across the network. It represents the live field of verified presence. *Example: The Presence Layer during a global solstice ritual may show thousands of active rituals and trails.*

Trail Divergence

→ A voluntary decision to stop recognizing or coordinating with another handle. Divergence can be silent or symbolic. *Example: A handle may exclude another from a future ritual based on visible trail behavior.*

Trail Zero

→ The original, immutable memory anchor of PresenceNet. It includes the first Marks, rituals, and Completion events that define the Genesis of the network. *Example: Trail Zero may contain the Timekeeper’s Oath and the Genesis Circle Ritual.*

SECTION 4 — SYSTEM PRIMITIVES

4.1 — Mark

A **Mark** is the atomic action in PresenceNet — a cryptographically signed, timestamped declaration of presence. It is sovereign, irreversible, and self-validating.

Fields

- sender_handle (*string*) — Handle of the initiator
- recipient_handle (*string | null*) — Target of the mark; null if self-mark
- timestamp (*ISO 8601 UTC*) — Time of issuance

- `ritual_id` (*string, optional*) — Ritual this mark participates in
- `completion_id` (*string, optional*) — Links this mark to a completed ritual
- `biometric_flag` (*bool*) — Indicates if biometric confirmation was used
- `memo` (*string, optional*) — Encrypted, optional note
- `tx_hash` or `record_id` (*string*) — Reference to ledger or data store

Rules

- A valid mark **must be signed** using the sender's private key
- Marks are **append-only and final** — they cannot be revoked or edited
- Self-marks must be routed via a **witness or designated public receiver**
- Marks default to **private**; visibility is opt-in
- Only the **sender** can issue a mark

4.2 — Trail

A **Trail** is the ordered, append-only memory ledger of a handle. It contains all marks sent and received, and represents a sovereign participant's presence history.

Rules

- Trails are **immutable** and **chronologically ordered**
- Each entry references a valid Mark
- Only the trail owner can access their **full private trail**
- Users may **publicly disclose** specific entries or subsets
- Trails are used for:
 - **Ritual eligibility**
 - **Vault access**
 - **Identity emergence**

4.3 — Ritual

A **Ritual** is a structured coordination pattern involving time-bound participation and defined completion rules. Rituals enable shared presence and conditional memory.

Fields

- `ritual_id` (*string, unique*) — Global ritual identifier
- `host_handle` (*string*) — Creator and coordinator of the ritual
- `start_time` / `end_time` (*ISO 8601 UTC*) — Ritual window
- `required_participants` (*list, optional*) — Handles required for coordination
- `required_marks` (*object, optional*) — Schema of expected marks

- `completion_condition` (*expression*) — Logical condition required for completion
- `reward_vault_id` (*string, optional*) — Vault unlocked upon completion
- `public_visibility` (*bool*) — Whether participation is publicly visible

Rules

- Only marks made within the ritual window **count toward completion**
- Rituals may be:
 - **Solo** (self-marking)
 - **Pairwise** (e.g. initiator + responder)
 - **Group-based** (multi-participant thresholds)
- A **Completion Mark** is only issued if `completion_condition` is met
- Rituals may unlock:
 - Vaults
 - Chambers (if advanced-tier)
 - Continuation rituals

4.4 — Vault

A **Vault** is a multi-user, threshold-based coordination space for storing assets, symbolic access, or collective memory. Vaults may be financial, symbolic, or purely ritual in nature.

Fields

- `vault_id` (*string, unique*) — Global vault identifier
- `members` (*list*) — Handles authorized to participate
- `signing_threshold` (*object*) — Required number of approvals (e.g., 2 of 3)
- `access_rules` (*object*) — Conditions required to join or act (e.g., “must complete ritual X”)
- `vault_type` (*string*) — financial, symbolic, or memory
- `created_at` / `modified_at` (*timestamps*) — Lifecycle metadata

Rules

- A vault **must define a purpose** at creation
- All actions (e.g. send, unlock, update) require **threshold-based multi-signature approval**

- **Symbolic vaults** may not exist on-chain, but must be **verifiable through presence**
- Vaults can be linked to rituals, trails, or chambers for advanced coordination

4.5 — Completion Mark

A **Completion Mark** is a special type of Mark issued after a ritual's conditions have been fulfilled. It confirms coordinated presence and may unlock access to other system elements.

Fields

- sender_handle (*string*) — Ritual host or protocol system identity
- recipient_handle (*string*) — Participant who completed the ritual
- ritual_id (*string*) — Reference to the completed ritual
- timestamp (*ISO 8601 UTC*) — Moment of issuance
- confirmation_data (*object, optional*) — Signature list, quorum proof, or audit trail

Used to:

- Unlock **new rituals, vaults, or chambers**
- Serve as **proof of coordinated achievement**
- Anchor **irreversible shared moments** in trail memory
- Determine **eligibility for sealed spaces** (e.g. Chamber access)

4.6 — Chamber

A **Chamber** is a sealed, sovereign memory container generated upon completion of an advanced ritual. It is accessible only to verified participants and serves as a permanent structure within the ritual memory layer.

Fields

- chamber_id (*string, unique*) — Global chamber identifier
- ritual_id (*string*) — Ritual that created the chamber
- participants (*list*) — Private list of eligible handles
- public_marks (*list*) — Opt-in marks marked as publicInChamber: true
- content (*object*) — Text, media, or structured symbolic memory
- unlocks (*object, optional*) — Linked ritual_id or vault_id
- created_at (*timestamp*) — Time of formation

Rules

- Chambers are **only created by rituals of tier: “advanced”**
- Chambers are **private by default** — only visible to qualifying participants
- Public presence within a chamber is **opt-in** via publicInChamber
- **Completion Marks must be issued** before chamber access is granted
- Chambers are **immutable** — they **cannot be deleted**
- Chambers may contain:
 - Public mark wall (optional)
 - Trail links
 - Ritual continuations

4.7 — Witness

A **Witness** is any handle that co-signs or receives a Mark, Ritual, or Vault action for the purpose of attestation, symbolic anchoring, or coordination. Witnessing is a **protocol role**, not a standalone primitive.

Used For

- Attesting to presence
- Participating in multi-signature rituals
- Enabling coordinated Vault actions
- Hosting self-marks
- Enriching Trails through social acknowledgment

Self-Mark Witnessing

Because users cannot mark their own handle, **self-marks are routed to a designated witness address** (e.g., \$witness).

Rules

- The recipient_handle is set to the witness address
- The mark must still be signed by the sender
- A self_mark flag may be included
- The witness does not validate the mark — it hosts it
- Self-marks appear in the sender’s trail and may be publicly surfaced
- Witnesses enable sovereign self-marking without violating core presence rules.

SECTION 5 — SYSTEM BEHAVIOR & FLOW

5.1 — Sending a Mark

Process:

1. User opens SendMark interface
2. Inputs recipient handle (or leaves blank for system-routed self-mark)
3. Optionally selects a ritual or completion context
4. Biometric confirmation (enabled by default)
5. Mark is signed using user's private key
6. Mark is recorded in trail and optionally propagated to recipient
7. Notification (optional) sent to recipient or ritual network

Constraints:

- Sender must be authenticated
- Timestamp is generated at moment of signature
- Duplicate marks within the same ritual window may be rejected
- Biometric requirement may be toggled, but strongly recommended for rituals

5.2 — Trail Generation and Access

- Each handle has one Trail
- The trail is updated with every Mark sent or received
- By default, trails are private (encrypted and viewable only by the owner)
- Users may publish portions of their trail (public memories, ritual completions)
- Trail entries include:
 - timestamp
 - counterparty (if any)
 - ritual reference (if any)
 - mark integrity status
 - optional metadata: witness status, biometric flag

Trail queries may support:

- filter by ritual
- filter by person
- chronological view
- symbolic/graphical views (optional)

5.3 — Ritual Participation

1. User discovers or is invited to a Ritual
2. Sees start/end time and required action (e.g. "Mark \$host between 6PM–8PM")
3. Marks host or completes required trail actions

4. PresenceNet verifies:
 - validity of mark
 - timing window
 - optional biometric flag
 - witness conditions (if any)
5. Upon completion, user receives Completion Mark
6. Ritual memory added to trail

If Ritual includes a Vault unlock:

- Access rules processed
- Vault becomes visible/unlocked per policy

5.4 — Vault Creation and Approval

- Initiator selects:
 - Vault name
 - Purpose (symbolic, financial, memory)
 - Member handles
 - Threshold (e.g. 3-of-5)
 - Optional presence requirements (e.g., must have completed Ritual X)
- Vault is created and logged
- Any proposal (e.g., withdraw asset, update purpose) requires signatures
- Vault entries become part of trail if significant

5.5 — Receiving a Mark

- If recipient is another user:
 - Notification appears: “You were marked by \$sender”
 - Optionally accept or acknowledge
 - Trail entry is created with mark metadata
- If recipient is a vault or ritual host:
 - Mark is evaluated per protocol rules
 - Used to compute completion, access, or unlock logic

5.6 — Opting Out, Deactivation, and Trail Export

- Users may deactivate handle but cannot delete trail
- Export functionality must be available (plaintext, JSON, or other)
- Option to download encrypted copy of trail with signature proof

- If system supports witness address: user may “burn” final mark to indicate exit

SECTION 6 — STORAGE & DATA MODELS

6.1 — Mark Storage

- Marks may be stored:
 - On-chain (e.g., XRPL memo data or custom ledger)
 - In decentralized storage (e.g., IPFS, Arweave)
 - In encrypted private trail store (e.g., Firestore, local encrypted DB)
- Each mark should contain:
 - hash of content
 - signature of sender
 - timestamp
 - recipient or ritual target
 - optional biometric flag

6.2 — Trail Storage

- Trails are append-only logs
- May be stored privately (user-only), shared (ritual view), or publicly disclosed
- Trail entries must contain:
 - Reference to original mark (hash or ID)
 - Mark metadata (sender, recipient, context)
 - Verification status
 - Storage location (on-chain or off-chain)

6.3 — Ritual Definitions

- Ritual schemas must be signed by creator
- Ritual metadata includes:
 - ID, host, start/end time, required actions
 - Visibility (public/private)
 - Vault output, if any

6.4 — Vault Storage

- Vault metadata:

- vault_id
- members and keys
- type
- threshold
- presence-based access rules
- Stored as signed config files or smart contracts depending on deployment

6.5 — Cryptographic Models

- Ed25519 or ECDSA (for compatibility with XRPL and similar ledgers)
- Optionally include biometric confirmation hashes (never raw biometric data)
- All marks must be signed by sender
- Trails must validate mark integrity on access

SECTION 7

Perfect. To preserve the flow and deepen the structural foundation, the **Presence Hierarchy** should be integrated as a formal **Section 7.0 — Ontology of Presence**, serving as a structural preface to the Ritual Engine.

This allows 7.1 to remain focused on coordination theory, while 7.0 clearly establishes the **layered progression of meaning and memory** that everything else is built on.

✓ **Here's the complete new**

Section 7.0 — Ontology of Presence

7.0 — Ontology of Presence

The seven foundational layers of symbolic coordination in PresenceNet

PresenceNet is not a feature set. It is an ontological stack — a structured hierarchy of memory that begins with individual presence and builds toward symbolic governance. Each layer is emergent, composable, and sovereign. Nothing is skipped. Nothing is coerced. Each layer makes the next one possible.

This is not a UI. It is the structural reality of presence in protocol form.

Layer	Name	Description
0	Presence	The foundation. A participant exists. Nothing is marked yet, but the capacity to mark exists.
1	Mark	The atomic signal of presence. A deliberate, signed, timestamped declaration. Immutable.
2	Trail	The ordered memory of all valid marks. Sovereign and append-only. Identity begins here.
3	Ritual	Structured coordination. Marks gain timing, logic, and meaning. Presence becomes intentional.
4	Chamber	Sealed, shared memory formed after advanced rituals. A symbolic artifact of presence.
5	Vault	Multi-user threshold coordination. Presence-based control over assets, memory, or symbolic access.
6	Coordination Field	The full, living system of interlinked marks, trails, rituals, chambers, and vaults. A decentralized symbolic architecture for shared action.

Design Properties of the Hierarchy

- **Progressive:** Each layer emerges naturally from the previous one

- **Non-coercive:** No layer is required to access the one above
- **Verifiable:** Every transition between layers is backed by signed memory
- **Sovereign:** Users retain control at every layer
- **Symbolic:** Each layer encodes deeper meaning — not just utility

This is PresenceNet's memory architecture. It is not enforced by code, but respected by design. It is what allows presence to be more than a status — it becomes a substrate for ritual, access, and shared memory.

Why It Matters

Without this hierarchy, coordination becomes arbitrary.

With it, coordination becomes sacred.

The protocol doesn't ask:

"What did you do?"

It asks:

"Where have you been — and who remembers with you?"

This structure makes it possible to mark, remember, and build without surveillance, scoring, or centralization.

This is the Ritual Engine's foundation.

Presence first. Then memory. Then meaning. Then coordination.

Ready to enter Section 7.1 — Coordination Theory?

Great.

Here is the full draft of **Section 7.1 — Coordination Theory**, written in full alignment with the sacred structure and laws of PresenceNet.

7.1 — Coordination Theory

A framework for voluntary, presence-based coordination in decentralized space

♦ Definition

In PresenceNet, **coordination is the act of aligning presences through shared structure — not shared pressure.**

It is not messaging.

It is not nudging.

It is not engagement.

Coordination is the emergence of shared timing and intent through voluntary marks, structured rituals, and memory-linked access.

No one is required to show up.

But when they do, it is final — and it is real.

♦ Core Premise: Coordination Without Coercion

PresenceNet rejects coercive coordination — systems that extract participation through:

- Notifications
- Likes
- Leaderboards
- Calendar prompts

- Reputation scores

These are tools of performance, not presence.

Instead, PresenceNet offers **sacred structure**:

If a person chooses to align, the system **holds their presence in memory — immutably and with dignity**.

If not, **their silence is respected**.

There is no penalty for divergence.

Only memory of those who chose to mark.

♦ **Three Forms of Coordination**

PresenceNet recognizes three distinct forms of coordination, each with increasing intentionality:

Type	Definition	Mechanism	Example
Implicit	Ambient alignment without direct connection	Shared ritual timing, seasonal recurrence	Solitude marks at midnight every full moon
Explicit	Structured, opt-in alignment across handles	Rituals with time windows and participation logic	Circle Ritual with 5 invited participants
Symbolic	Meaning encoded through memory and structure	Completion Marks, Vaults, Trail branching	Completing Guardian unlocks access to Vault of Elders

All three are valid.

All three are sovereign.

Coordination in PresenceNet is a **layer of memory**, not an interface of control.

♦ **Trail-Based Coordination**

The Trail is the substrate of coordination.

In PresenceNet, you do not coordinate through chat, likes, or votes.

You coordinate through **presence recorded in trail**.

Every Mark contributes to:

- Eligibility (Did you show up before?)
- Symbolic weight (How often have you marked?)
- Path selection (Which rituals have you completed?)
- Vault unlocks or locks

There is no public reputation layer.

There is only the trail — and **the truth it tells**.

♦ **Divergence is Not Failure**

In many systems, coordination is enforced:

- “If you didn’t show up, you lose.”
- “If the quorum isn’t met, the group fails.”
- “If the engagement drops, the feature is killed.”

PresenceNet says:

“If the ritual is not completed, it becomes memory — not failure.”

Unfinished rituals become:

- Expired invitations
- Faded trail entries
- Signals of disalignment — not punishment

This is **coordination through availability**, not demand.

♦ **Completion is Not Consensus**

Ritual completion does not mean agreement.

It means presence was verified under shared structure.

There is no voting mechanic in PresenceNet.

There is only:

- **Marks made**
- **Conditions met**
- **Access unlocked**

Consensus is not declared — it is **embodied** through shared trail truth.

♦ **Scaled Coordination, Sovereign Structure**

PresenceNet can scale from:

- A solo daily ritual
- To a 5-person Circle

- To a 10,000-person global Solstice Anchor

And it never requires:

- Central scheduling
- Mass messaging
- Forced sync

Rituals, by design, are **independent objects of structure**.

Builders can reference, copy, or chain them.

Participants can ignore or enter them.

Either choice is held in the trail.

♦ **Final Principle**

Coordination in PresenceNet is not about making people do things together.

It is about **offering structure** in which people may — if they choose — align.

And if they do, they become memory.

And if they don't, they remain sovereign.

End of Section 7.1

Ready to proceed to **7.2 — Ritual Templates** when you are.

Excellent. Here's the full draft of **Section 7.2 — Ritual Templates**, written in alignment with the philosophy and structural law of PresenceNet.

7.2 — Ritual Templates

A canon of structured coordination forms in the PresenceNet protocol

♦ Purpose

A **Ritual Template** is a predefined structure for coordinated presence. It defines:

- The number of participants
- The time window
- The completion conditions
- The symbolic or structural outcome

Rituals are **opt-in containers of alignment** — not challenges, not prompts.

Templates are not gamified actions. They are **voluntary vessels for meaning**.

A template offers a form. A participant offers presence. The result is memory.

♦ Design Principles

Every Ritual Template in PresenceNet must:

- Respect silence
 - Avoid coercion
 - Require no ongoing attention
 - Function whether it completes or not
 - Be meaningful whether public or private
 - Create **memory, not engagement**
-

♦ Foundational Ritual Templates (Tier 1)

These five rituals form the core grammar of PresenceNet coordination:

Name	Type	Purpose
Initiation Ritual	Solo	Anchor the first mark of a new trail
Witness Ritual	Pairwise	Confirm mutual presence across handles
Circle Ritual	Group	Time-bound coordination with a threshold
Memorial Ritual	Open	Honor a moment, person, or transition
Unlock Ritual	Conditional	Grant access to vaults or content based on trail state

Each foundational ritual can be extended or modified with **mechanics** (7.3) and **logic** (7.4).

♦ **Advanced Ritual Templates (Tier 2)**

These rituals expand the symbolic and social expressiveness of PresenceNet:

Name	Type	Purpose
Echo Ritual	Network Ripple	Send presence outward from a single origin

Witness Chain	Sequential	Form a continuous chain of verified presence
Reflection Ritual	Memory Reinforcement	Revisit and deepen past rituals
Presence Lock	Threshold Unlock	Delay access until presence quota is reached
Anchor Ritual	Temporal/Spatial	Fix a moment in time or place with high precision
Solitude Ritual	Personal	Mark presence without interaction
Split Ritual	Divergence	Create branches of future trail based on choice
Time Capsule Ritual	Delayed Reveal	Lock memories or content for future opening
Guardian Ritual	Witness-Required	Require validation by a trusted entity to proceed

◆ **Template Format**

Each template includes:

- Name & Version

- Type (solo, pairwise, group, etc.)
- Purpose
- Participation Rules
- Completion Condition
- Result
- Notes

This structure maintains clarity for implementers, while allowing extensibility through mechanics and logic.

♦ **Template Sovereignty**

Ritual Templates are:

- **Optional** — they are examples, not mandates
- **Composable** — templates may be chained, forked, or evolved
- **Sacred** — a template is not an app feature, it is a container for human presence

Apps may define custom rituals, but all templates must comply with:

- Section 2: Core Philosophy
- Section 3: Glossary
- Section 7.3–7.4: Mechanics and Logic

♦ **Example: Circle Ritual**

Name: Circle Ritual

Version: v1.0

Type: Group Coordination

Purpose: Time-bound collective presence among multiple participants

Participation Rules:

- Host: defines participants (3–7 recommended)
- Ritual window: fixed (e.g., 2 hours)
- Threshold: minimum 3 marks

Completion Condition:

- At least 3 participants mark the host within the window

Result:

- All qualifying participants receive a Completion Mark
- Trail entries show presence, timing, and threshold met

♦ Trail Impact

Each ritual contributes to:

- Memory density (repetition, reflection, echo)
- Trail branching (divergence, split)
- Unlocks (access to vaults, chambers, or next rituals)
- Symbolic progression (presence-based growth)

The **Template Layer** is the bridge between ritual design and ritual behavior.

It names the patterns through which presence may align.

End of Section 7.2

Ready to proceed to **7.3 — Ritual Mechanics** when you are.

Excellent. Here is the full draft of **Section 7.3 — Ritual Mechanics**, written in sacred alignment with PresenceNet's architecture and principles.

7.3 — Ritual Mechanics

The behavioral layer that governs how rituals evolve, replicate, and dissolve

♦ What is a Ritual Mechanic?

A **Ritual Mechanic** is a **non-user-facing protocol rule** that determines how a ritual behaves over time.

Where **Ritual Templates** define structure, **Mechanics** define motion.

They are modifiers — not creations.

Templates are nouns.

Mechanics are verbs.

They govern the life cycle, flow, and transformation of rituals.

These are not features or effects. They are sacred architectural tools — how rituals loop, mutate, pass on, or end.

♦ Design Principles

All Ritual Mechanics must:

- Respect voluntary presence
- Avoid gamification
- Never obscure finality (a mark is a mark)
- Preserve memory above engagement

- Enable symbolic depth without performance

♦ **Core Mechanics (Layer 1)**

Mechanic	Function	Used For
Recursion	Ritual restarts automatically after completion	Habits, devotion, streaks
Mutation	Ritual transforms into another upon completion	Quests, leveling, initiations
Delegation	Completion rights can be passed to another	Guardianship, legacy
Replication	Ritual spawns copies upon completion	Viral rituals, movements
Merge	Two or more rituals combine into a new one	Synthesis, fusion, shared ceremonies
Decay	Ritual becomes invalid after time passes	Impermanence, urgency
Reversal	Ritual may undo prior memory via sacred counteraction	Healing, forgiveness

Weighting	Ritual contributes symbolic weight to trail	Reputation without score
Latency	Ritual delays completion after final mark	Reflection, suspense
Sealing	Ritual locks forever after X completions	Scarcity, rare moments

Each mechanic is optional and must be declared in the ritual definition using its appropriate field.

◆ **Field-Level Definitions**

Mechanic	Field	Example Value
Recursion	recurring: true + interval: 7d	Weekly Solitude Ritual
Mutation	on_completion_morph: ["ritual_id"]	Circle evolves into Guardian
Delegation	completion_delegate: "\$heir"	Only heir may complete
Replication	replicates_on_completion: true	Echo Ritual spawning
Merge	merge_trigger: ["ritualA", "ritualB"]	A + B = C

Decay	decay_after: "48h" + on_decay: "trail_faded"	Expiring rituals
Reversal	requires_reversal_path: true	Undo past vault
Weighting	ritual_weight: 50	Symbolic identity building
Latency	delayed_completion: true, delay_duration: "7d"	Time Capsules
Sealing	max_completions: 14, auto_seal: true	Genesis rituals

◆ Composability

Mechanics can be layered. For example:

- A **Recurring + Weighted + Latent** ritual may be used for spiritual journaling.
- A **Sealed + Merged + Reversal** structure could form a legacy vault system.

Mechanics are **pure behavior** — they do not change access logic or eligibility.

That is handled in **7.4: Ritual Logic**.

◆ Example: Recursion

Ritual: Midnight Anchor

Fields:

recurring: true

interval: 24h

ritual_window: "00:00–00:10"

This ritual restarts every night. Presence must be marked again, or not at all.

Memory builds over time, not through points — but through consistency.

♦ Symbolism Through Mechanics

The deeper function of Ritual Mechanics is **to give presence form beyond time**.

They allow memory to:

- Evolve
- Decay
- Recur
- Be undone
- Be made scarce

Presence becomes programmable.

Not for automation — but for **ritual truth**.

♦ Implementation Note

All mechanics must be processed at the **protocol level or trusted app layer**.

They must not rely on user behavior or UI artifacts.

They must not create unprovable outcomes.

If a ritual decays, seals, mutates, or spawns — the **ledger or trail must reflect it**.

Ritual Mechanics are the verbs of PresenceNet.
They give life to form — and form to life.

End of Section 7.3

Ready to continue with **7.4 — Ritual Logic** when you are.

Perfect — thanks for the clarification. Here's the **final version of Section 7.4 — Ritual Logic**, using clean text structure, light bullets, and tables where appropriate. Tone is technical, direct, and implementation-aligned, with minimal spacing.

7.4 — Ritual Logic

Ritual Logic defines the conditions under which a participant is allowed to complete a ritual. It is the programmable access layer of PresenceNet, evaluated at the time a mark is attempted. It determines eligibility based on the participant's trail—cryptographically signed, immutable records of past presence.

Ritual Logic is not the same as Ritual Mechanics. Mechanics define how a ritual behaves over time. Logic determines who can enter and when. These two layers can coexist, but are functionally separate.

Logic checks are deterministic. They return true or false based solely on the participant's memory. If true, the ritual may be completed. If false, the ritual remains inaccessible—quietly and without penalty.

Core Logic Primitives

Condition	Description
<code>completedRitual(id)</code>	Participant has a valid Completion Mark for the specified ritual ID
<code>hasMarkFrom(handle)</code>	Trail includes a mark received from the specified handle
<code>hasBiometricMarks(count)</code>	Trail includes at least count marks with <code>biometric_flag = true</code>
<code>withinTrailDepth(min)</code>	Total number of trail entries meets or exceeds a minimum threshold

<code>geofencedMark(lat, long, r)</code>	At least one mark was sent from within r km of the provided coordinates
<code>timeSinceLastMark(op, d)</code>	Time since last mark satisfies condition (e.g., >7d, <24h)
<code>customBooleanExpression()</code>	Advanced composable logic expression using AND, OR, NOT, nested logic

Examples

- `completedRitual("guardian")`

Only participants who have completed the Guardian ritual may proceed.

- `hasMarkFrom("$selena")`

The participant must have been marked by \$selena at least once.

- `withinTrailDepth(50)`

Trail must contain 50 or more marks. Used for elder-only access.

- `geofencedMark(48.8566, 2.3522, 10)`

Presence must be verified within 10km of Paris coordinates.

- `customBooleanExpression("(completed.guardian && biometric) || hasMarkFrom.$mentor")`

Complex access logic using compound trail conditions.

Evaluation Rules

- All logic is based solely on the participant's verified trail.

- There is no central validation or scorekeeping.
- Failed logic conditions do not generate logs or penalties.
- Trails remain sovereign—even when logic denies entry.

Implementation Considerations

- Logic may be evaluated client-side, server-side, or on-chain.
- Trails must be accessible to the logic engine for verification (full or partial depending on visibility settings).
- No ritual logic should depend on mutable off-chain state, social consensus, or token holdings.

Extensibility

Ritual logic is schema-based and open-ended. New primitives may be added as long as they are:

- Deterministic
- Verifiable from the trail
- Consistent with PresenceNet's core philosophy (Section 2)

Future primitives may include:

- `hasNotCompleted(ritual_id)` — Inverse ritual condition
- `markedDuring(timestamp_range)` — Temporal condition
- `coMarkedWith(handle)` — Shared presence with another participant
- `silentFor(duration)` — Silence as a logic constraint

Final Principle

Ritual Logic is memory-based access control. It does not measure worth, only alignment. There is no reputation. No judgment. Just trail-based truth.

PresenceNet uses logic to protect the sacred entry into coordination—not to score or sort participants. Those who meet the memory conditions may proceed. Those who do not remain sovereign.

Ready for **7.5 — Ritual Vault Integration?**

Excellent. Here's the full draft of **Section 7.5 — Ritual Vault Integration**, written with precision, clarity, and full adherence to PresenceNet protocol design.

7.5 — Ritual Vault Integration

PresenceNet Vaults are multi-user, multi-signature XRPL wallets governed by presence-based rules. Rituals serve as the programmable access and coordination layer for these Vaults. Rituals may unlock Vaults, submit qualifying marks as signatures, or serve as structural prerequisites for Vault actions.

This section defines the integration patterns between rituals and Vaults, including access control, signature thresholds, memory anchoring, and governance logic.

Integration Modes

PresenceNet supports five core ritual-to-vault integration patterns. These modes are composable and may be combined within a single Vault configuration.

Integration Type	Description	Use Cases
Ritual-as-Key	Completion of a ritual unlocks access to a Vault	Onboarding, initiation, symbolic unlocks

Ritual-as-Signature	Each qualifying ritual mark counts as a valid signature for Vault action	Group approval, coordinated funding
Ritual-as-Condition	Vault remains locked unless specific ritual conditions are met	Hierarchies, lore access, sacred gating
Ritual-as-Artifact	Rituals write symbolic data into the Vault memory, not unlock it	Journals, dedications, collaborative scrolls
Ritual-as-Governor	Ritual completion is required to change Vault rules or configuration	Governance, multi-party rule updates

1. Ritual-as-Key

- Completion of a designated ritual grants Vault access.
- Can be permanent or time-bound, as defined in the ritual schema.
- Vault stores a reference to the required ritual(s) for access evaluation.

Example:

Complete the “Initiation Ritual” → gain access to \$circlevault.

Implementation Notes:

- Vault system checks for `hasCompleted(ritual_id)` in participant trail.
 - May be extended with additional trail logic (e.g. biometric, witness).
-

2. Ritual-as-Signature

- Marks made during a ritual count as multi-sig signatures toward a Vault proposal.
- Rituals function as structured multi-party authorization flows.

Example:

All 3 mentors mark a “Release Ritual” → threshold met → funds disbursed.

Implementation Notes:

- Vault proposal object includes expected ritual_id and signer handles.
 - Ritual engine verifies valid mark timestamps and authenticity.
 - Signatures expire based on ritual window or timeout condition.
-

3. Ritual-as-Condition

- Vault is sealed unless specific rituals have been completed.
- Presence history becomes the key, not participation in a current ritual.

Example:

“Vault of the Ancients” unlocks only for users who have completed:

- Initiation
- Guardian
- Memorial

Implementation Notes:

- Logic is enforced using requiresCompletionOf and trail state.

- Access evaluation is read-only and deterministic.
-

4. Ritual-as-Artifact

- Ritual writes symbolic or narrative data into a Vault's memory field.
- No access is granted or gated. The Vault becomes a living archive.

Example:

Solitude Ritual writes encrypted reflections to a shared memory Vault.

Implementation Notes:

- Ritual Completion Mark includes encrypted payload.
- Vault record includes `ritual_log[]` of accepted completions.

Note: In v0.2+, this function may migrate to Chambers unless financial coordination is required.

5. Ritual-as-Governor

- Ritual completion is required to change Vault configuration.
- This replaces admin rights with presence-based consent.

Example:

To change threshold from 3-of-5 to 4-of-5, participants must complete "Consensus Ritual".

Implementation Notes:

- Vault action proposal includes required `ritual_id`.

- Participants must complete the ritual before the update executes.
 - Optional: Vault records symbolic record of governance action in trail.
-

Design Rules

- Vaults must declare their ritual dependencies at creation.
 - Ritual completions used for Vaults must be verifiable via trail.
 - No ritual may bypass signing thresholds unless explicitly defined as such.
 - Vaults may chain to Chambers, Rituals, or other Vaults based on ritual state.
-

Example Use Case: Multi-Layered Vault Unlock

- Participant completes Circle Ritual → gains access to \$groupvault (Ritual-as-Key)
- Funds are released after 3 participants complete Witness Ritual → each mark counts as signature (Ritual-as-Signature)
- Vault structure change is only allowed if a Consensus Ritual is completed by all current members (Ritual-as-Governor)

This structure allows complex, symbolic, and sovereign coordination with no off-chain trust dependency.

Final Note

Ritual Vault Integration turns memory into access. There is no admin role. No token gate. Only presence—proven through coordinated ritual, held in the trail, and interpreted by the protocol.

Ready to continue with **7.6 — Ritual Creation Interface**?

Excellent. Here is the full draft of **Section 7.6 — Ritual Creation Interface**, presented in a clear, implementation-ready format while remaining consistent with the PresenceNet design philosophy.

7.6 — Ritual Creation Interface

The Ritual Creation Interface is the method by which developers or users define, configure, and publish rituals within PresenceNet-compatible applications. It produces a structured object or schema that can be interpreted by the Ritual Engine and validated against incoming marks.

This section defines the required fields, supported modifiers, and structural best practices for creating rituals. It also establishes the boundary between interface flexibility and protocol enforcement.

Core Structure of a Ritual Definition

Each ritual must contain the following core fields:

- `ritual_id` (string): Globally unique identifier
- `host_handle` (string): Creator or initiating identity
- `ritual_type` (enum): One of solo, pairwise, group, open, or conditional
- `ritual_window` (object): Start and end timestamps in ISO 8601 format
- `participant_handles` (optional array): List of expected participants (null if open)
- `completion_condition` (object): Logic expression or rule set required to issue Completion Mark
- `public_visibility` (boolean): Determines whether ritual activity is publicly viewable

- **biometric_required** (boolean): If true, marks must include biometric confirmation
- **memo_allowed** (boolean): If true, participants may attach encrypted notes
- **vault_linked** (optional string): Vault ID to be unlocked or modified by completion
- **mechanics** (optional object): See Section 7.3 for full structure
- **logic** (optional object): See Section 7.4 for supported logic schema

These fields form the minimum required structure for a valid ritual definition.

Applications may extend this schema with metadata or additional display fields, but the above must be preserved and valid.

Creation Flow (UI or Code)

1. **Choose Template (optional):** Select a base ritual type (e.g. Circle, Solitude)
2. **Define Participants:** List handles or leave open
3. **Set Time Window:** Absolute time range for ritual activity
4. **Specify Logic:** Trail-based access conditions
5. **Add Mechanics (if any):** Recursion, decay, mutation, etc.
6. **Link Vaults or Continuations:** Specify what is unlocked or created
7. **Publish:** Ritual is made available and its ID is distributed

A valid ritual object must be signed by the host handle and optionally registered in a public or private ritual directory.

Programmatic Schema Example (Minimal)

```
{
```

```
"ritual_id": "circle_spring2025",
"host_handle": "$selena",
"ritual_type": "group",
"ritual_window": {
  "start": "2025-03-21T12:00:00Z",
  "end": "2025-03-21T14:00:00Z"
},
"participant_handles": ["$mayo", "$dahlia", "$rain"],
"completion_condition": {
  "threshold": 3
},
"public_visibility": true,
"biometric_required": false,
"memo_allowed": true
}
```

Advanced Schema Example (With Logic & Mechanics)

```
{
  "ritual_id": "guardian_passing",
  "host_handle": "$council",
  "ritual_type": "conditional",
  "ritual_window": {
    "start": "2025-06-01T00:00:00Z",
    "end": "2025-06-30T00:00:00Z"
  },
  "participant_handles": null,
  "completion_condition": {
    "and": [
      { "completedRitual": "initiation" },
      { "hasMarkFrom": "$mentor" },
      { "trailDepth": { "gte": 50 } }
    ]
  },
  "mechanics": {
    "recurring": true,
    "interval": "1y"
  },
  "logic": {
    "requiresBiometric": true,

```

```
"geofencedMark": {  
  "lat": 34.0522,  
  "lon": -118.2437,  
  "radius_km": 10  
},  
"vault_linked": "vault_elderpath",  
"public_visibility": false,  
"memo_allowed": false  
}
```

Ritual Publishing Considerations

- Rituals should be timestamped and signed by the host for validity.
 - Ritual IDs must be unique within the namespace of the app or protocol layer.
 - Apps may offer “ritual registries” (public or private) for discoverability (see 7.8).
 - Rituals do not enforce UI—they define structure and rules, not appearance.
-

Best Practices

- Keep required fields minimal but expressive
 - Use templates for common rituals to reduce errors
 - Avoid using logic or mechanics unnecessarily if a simple ritual suffices
 - When chaining rituals, be clear about what each one unlocks or creates
 - Preserve the tone of PresenceNet: symbolic, sovereign, and memory-driven
-

Final Note

The Ritual Creation Interface gives developers and ritual authors the ability to define precise, memory-bound structures for coordination. It is not a gamification tool. It is a configuration space for sacred timing, symbolic action, and sovereign entry into the coordination layer of PresenceNet.

Ready to proceed to **7.7 — Ritual System Dynamics**?

Here is **Section 7.7 — Ritual System Dynamics**, written with a focus on high signal, architectural clarity, and true to the systemic and emergent design of PresenceNet.

7.7 — Ritual System Dynamics

Rituals in PresenceNet do not exist in isolation. They form a dynamic, interdependent layer that coordinates participants across time, memory, and symbolic meaning. This section outlines how rituals behave as a system—how they chain, cascade, evolve, and form a living coordination field without requiring central scheduling or communication.

The Ritual Engine is not a scheduler. It is a decentralized protocol for programmable presence. Its output is not attention—it is memory.

1. Ritual Chaining

Rituals can create logical or symbolic dependencies between one another. When a ritual is completed, it may:

- Unlock access to a new ritual (on_completion → ritual_id)
- Morph into a new ritual with updated structure (mutation)
- Update logic in another ritual (e.g., lowering a barrier after quorum is met)

These chains form symbolic sequences (e.g. Initiation → Guardian → Elder) and can represent narrative paths, learning arcs, or spiritual progression.

2. Ritual Cascades

A single ritual can trigger multiple downstream effects based on participation or completion.

Examples:

- Completing a global Solstice Anchor ritual spawns individual Reflection Rituals for each participant
- A Circle Ritual unlocks multiple Vaults for different threshold groups
- An Echo Ritual begins a public chain of presence across 3 or more levels

Cascades allow rituals to scale horizontally, increasing impact without increasing central complexity.

3. Recurrence and Seasonal Synchronization

Rituals may be designed to recur on fixed intervals using the recurring mechanic (e.g., weekly, monthly, yearly). This allows temporal rhythms to emerge across users without requiring real-time coordination.

Examples:

- A Solitude Ritual every Sunday at 6pm
- A Witness Chain that reactivates annually on the day it was born
- A Memorial Ritual that opens each year on a historic date

These patterns generate **ambient coordination**, where participants are aligned through shared time—without messaging or prompts.

4. Trail Density and Identity Emergence

The more rituals a participant completes, the more dense and symbolically rich their trail becomes. This memory density is not gamified—it carries no score—but it becomes a visible pattern of:

- Consistency (recurring rituals completed over time)
- Divergence (which paths or splits were taken)
- Silence (gaps in presence or chosen abstention)
- Specialization (which ritual types dominate the trail)

These emergent patterns allow symbolic roles to form (e.g., “Healer”, “Witness”, “Archivist”) based entirely on voluntary memory—not on assigned labels or tokens.

5. Collective Presence Patterns

Ritual data (visible marks, Completion Marks, vault unlocks) may reveal emergent presence behavior across the network. This is referred to as the Presence Layer—an active field of rituals, trails, and symbolic memory visible in aggregate but coordinated individually.

Examples:

- Hundreds of Witness Rituals activating in response to global events
- A Presence Lock ritual that unlocks a Chamber when 1,000 users mark in solitude
- A symbolic Vault that only opens after 50 users complete a Memorial sequence

These phenomena require no messaging, algorithmic push, or group chat. They emerge from structure, timing, and truth.

6. System Integrity Without Scheduling

There is no centralized calendar or scheduler in PresenceNet. Rituals are created and activated independently. Coordination happens because participants align voluntarily—not because the system enforces time slots or reminders.

- Each ritual defines its own window and logic
- Participants may choose to engage—or not
- Completion is only possible within valid conditions
- No ritual depends on messaging, chat, or live syncing

This allows the system to scale globally without requiring simultaneous activity.

7. Memory Propagation vs Communication

PresenceNet does not pass messages. It propagates memory.

When a participant completes a ritual:

- Their trail is updated with verifiable proof
- That memory can unlock access, contribute to thresholds, or spawn future rituals
- No message needs to be sent; no coordination channel needs to exist

This is memory-based coordination, not message-based coordination.

8. Rituals as Cultural Infrastructure

Over time, rituals form the cultural foundation of a PresenceNet-based society. They become the shared grammar of:

- Belonging
- Initiation
- Trust
- Healing

- Divergence
- Reunion

These patterns are not enforced. They are offered—and remembered.

Final Note

The Ritual Engine is not a calendar, nor a task system. It is a decentralized structure for symbolic coordination across time. It enables memory to become shared action—without performance, without noise, and without surveillance.

PresenceNet's system dynamics prove that you don't need a feed to feel connected. You need form. You need memory. You need sovereign, optional structure.

That is what the Ritual Engine provides.

Ready for 7.8 — Ritual Discovery Patterns?

Here is **Section 7.8 — Ritual Discovery Patterns**, written clearly and technically, framed as optional infrastructure for PresenceNet-compatible apps. This section respects PresenceNet's core philosophy while offering practical guidance for implementation.

7.8 — Ritual Discovery Patterns

PresenceNet does not mandate a global directory or feed for rituals. However, developers building PresenceNet-compatible applications may choose to expose rituals through a discovery layer. This section outlines recommended patterns for enabling discoverability while preserving privacy, sovereignty, and signal integrity.

Ritual discovery is not required by the protocol. It is an optional application-layer feature that supports coordination at scale by making available rituals visible to those who may wish to participate.

Discovery Models

There are three common discovery patterns, each serving different coordination needs:

1. Private Invitation

- Rituals are shared directly between known participants
- The `participant_handles` field is explicitly set
- Access to the ritual definition is restricted (e.g., encrypted, time-locked, invite-only)

2. Public Directory

- Rituals are listed in a public registry, either on-chain or app-managed
- Visibility is controlled via `public_visibility: true`
- Users can browse by tag, template type, or time window

3. Contextual Activation

- Rituals are suggested based on trail state or prior completions
- No feed is needed; the system simply checks if a user becomes eligible for a ritual and reveals it at the right time
- Example: A user completes “Initiation” → system reveals “Guardian” without browsing

Ritual Metadata for Discovery

To support voluntary discovery, rituals may include optional metadata fields:

- `title` (string): Short human-readable name
- `description` (string): Summary of ritual purpose

- tags (array): Descriptive keywords (e.g., “solitude”, “anchor”, “circle”)
- estimated_duration (string): Time expected to complete (e.g., “2h”, “3d”)
- language (string): Content language (for localized rituals)
- symbol_url (string): Optional icon or glyph reference
- visibility_scope (enum): "public", "private", "contextual"

These fields aid search and filtering but are not enforced by the protocol. They are ignored by the Ritual Engine and exist solely for UX.

Indexing Best Practices

- Public rituals should be signed and timestamped by their host
 - Listings should avoid algorithmic ranking, trending metrics, or attention scores
 - Filters should be opt-in: by time, type, theme, or known host
 - Ritual registries may be global or scoped to a specific app, network, or group
-

Privacy Considerations

- Public rituals may be discovered by anyone, but mark participation is still private unless opted in
 - Private rituals should never be exposed in public registries unless explicitly opened by the host
 - Contextual rituals should only be suggested if all access conditions are already met
-

Non-Feed Philosophy

PresenceNet is not a feed. It is a structure for sovereign coordination. Discovery patterns must not reintroduce social pressure, performance incentives, or engagement loops.

Rituals may be listed—but they must never be pushed.

Optional: Discovery Schema Example

```
{  
  "ritual_id": "anchor_solstice_2025",  
  "title": "Solstice Anchor",  
  "description": "Mark the longest day with a single presence at 12:00 UTC.",  
  "tags": ["anchor", "seasonal", "solitude"],  
  "visibility_scope": "public",  
  "estimated_duration": "5m",  
  "symbol_url": "https://rituals.presencenet.org/icons/anchor.svg",  
  "public_visibility": true  
}
```

Final Note

Discovery is a gateway to participation, not a demand for action. Rituals in PresenceNet should be easy to find—but never forced, gamified, or optimized for noise.

If the system is quiet, it is working.

Ready to continue with **7.9 — Ritual Memory Architecture?**

Here is **Section 7.9 — Ritual Memory Architecture**, written with precision and depth.

This section explains how ritual memory is recorded, visualized, and symbolically preserved within the PresenceNet protocol.

7.9 — Ritual Memory Architecture

Every completed ritual in PresenceNet leaves a permanent trace in the participant's trail. This trace is not a log of activity—it is a verified memory. Ritual Memory Architecture refers to how these memories are structured, surfaced, and used to build continuity, identity, and access within the protocol.

PresenceNet is a memory-first system. Rituals do not expire into nothing—they crystallize as memory, whether completed alone or with others. This section defines the architecture of ritual memory: how it is stored, what it unlocks, and how it can be made visible or symbolic.

1. Memory Anchoring

A completed ritual results in:

- A **Mark** (signed, timestamped presence)
- A **Completion Mark** (ritual-verified outcome)
- A **Trail Entry** (immutable memory record)

These entries are cryptographically signed and optionally reference:

- The ritual ID
- Biometric status
- Vault access changes
- Chamber unlocks
- Attached memos

- Temporal/geographic context

Together, this creates a **ritual memory anchor**: a verifiable, symbolic point in time.

2. Memory Density and Patterning

A trail with many ritual completions forms symbolic patterns, without scores or ranks. Patterns emerge through:

- **Frequency** (daily solitude, monthly witness)
- **Sequence** (initiation → guardian → split)
- **Type** (solo-heavy, witness-heavy, grief-heavy)
- **Gaping** (long silence followed by return)
- **Recurrence** (annual memorials or seasonal anchors)

These patterns are not externally judged. They are privately meaningful and optionally shared.

3. Visualization Possibilities

PresenceNet does not mandate a visual interface, but developers may choose to visualize ritual memory through:

- **Trail timelines** (chronological presence markers)
- **Symbol walls** (Completion Marks as glyphs or badges)
- **Memory glyphs** (visual summaries of ritual arcs)
- **Ritual scrolls** (structured group memory shared by all participants)
- **Chambers** (see Section 4.6 — sealed memory containers)

Each ritual entry may include optional symbolic metadata for rendering in a UI, but this is never required by the protocol.

4. Shared Ritual Memory

When multiple users complete a ritual together (e.g. Circle, Chain, Echo), each trail records individual marks and a shared Completion Mark. This creates distributed memory across participants.

Shared memory may also be:

- Stored in a symbolic Vault (ritual_log[])
- Linked in a Chamber
- Used to seed future rituals or reflect back in a Reflection Ritual

This forms the basis of **symbolic permanence** across coordinated action.

5. Personal Reflection and Reengagement

Users may voluntarily revisit ritual memory through:

- **Reflection Rituals** (re-marking past completions)
- **Silence** (deliberate distance from previous patterns)
- **Path Divergence** (choosing a split)
- **Repetition** (re-entering seasonal rituals with evolving meaning)

Ritual memory is not frozen. It becomes **cumulative, recursive, and interpretive**.

6. Storage and Persistence Rules

- All ritual memory is append-only and cannot be deleted.
 - Private rituals remain in the trail but are not publicly indexed.
 - Public Completion Marks may be rendered symbolically or as open proof-of-participation.
 - Memory should never be monetized, ranked, or tokenized by external systems.
-

7. Symbolic Functions of Memory

Ritual memory acts as:

- Proof of alignment
- Gate to access (Vaults, Chambers)
- Cultural belonging (e.g. presence in “Genesis 14”)
- Record of divergence (e.g. which Split path was chosen)
- Spiritual artifact (presence as artifact)

It replaces the need for credentials, likes, or performance logs. PresenceNet’s memory model treats presence as a **sovereign credential**—a record of where you were, and who you chose to be with.

Final Note

Ritual memory is the foundation of trail identity, symbolic access, and sovereign coordination. It is not data to analyze—it is proof to remember. In PresenceNet, every ritual creates presence. Every presence creates memory. And memory is sacred.

Ready to close Section 7 with **7.10 — Symbolic Governance via Rituals?**

Here is **Section 7.10 — Symbolic Governance via Rituals**, the final section in the Ritual Coordination Engine. This completes the ritual system with a forward-looking, technically-grounded model of presence-based governance.

7.10 — Symbolic Governance via Rituals

PresenceNet does not use tokens, voting shares, or delegated staking. Instead, it offers a governance model grounded in presence: a record of memory, alignment, and participation—verified through ritual.

This is not governance through reputation. It is governance through memory. Ritual-based governance enables collective decision-making, access control, and protocol change through trail-based proof, not performance metrics.

1. What is Symbolic Governance?

Symbolic governance is the ability to make meaningful decisions—protocolic, social, or structural—based on verified presence, without requiring tokens, platforms, or votes.

A symbolic governance action is not decided by:

- Wealth
- Activity
- Popularity

It is decided by:

- Completion of specific rituals
 - Valid marks under threshold logic
 - Sovereign memory recorded in trails
-

2. Governance Actions Enabled by Rituals

PresenceNet supports the following governance patterns via ritual:

- **Vault Configuration:** Changing a Vault's threshold, membership, or access logic only after completing a Consensus Ritual (see 7.5).
- **Ritual Authoring Rights:** Granting permission to define future rituals based on Completion Marks from previous governance paths.
- **Membership Updates:** Joining or exiting symbolic groups (e.g., Circle of Elders) via Completion Marks from rite-of-passage rituals.
- **Protocol Upgrades:** Triggering structural changes (e.g. new mechanics, registry formats) only after global symbolic coordination.

Each action is initiated through ritual logic, completed through coordinated marks, and validated via trails—not via consensus votes.

3. Example: Elder Vault Threshold Change

- Vault ID: \$councilvault
- Current Rule: 3-of-5
- Change Request: Move to 4-of-6
- Governance Ritual: circle_of_consensus
- Logic: Requires 6 participants with:
 - completedRitual("elder_initiation")
 - trailDepth $\geq$ 100
 - hasMarkFrom("\$founder")
- Completion: Each participant marks the ritual within a 72-hour window
- Result: Vault updates configuration automatically after threshold met

There is no token, no vote tally, no off-chain coordination required.

4. Design Considerations

- Ritual-based governance is memory-enforced. It cannot be forged or simulated.
 - Decisions are binding when trail conditions are verifiably met.
 - Governance rituals may include sealing mechanics (max_completions) or decay windows to enforce urgency or scarcity.
 - Trails offer full auditability of who participated and when.
-

5. Delegation Without Tokenization

Delegation can occur symbolically using Ritual Mechanics:

- completion_delegate allows a user to pass governance authority to another via ritual
- guardian_witness roles can be assigned through prior rites
- Legacy can be formalized through Completion Marks, not legal constructs

This enables inheritance, guardianship, or custodianship—anchored in presence, not contracts.

6. Limitations and Philosophy

PresenceNet governance is **symbolic, slow, and sacred**. It is not intended for high-frequency parameter tuning. It is best suited for:

- Foundational upgrades

- Group access control
- Ritual authorship rights
- Symbolic status transitions
- Vault and Chamber evolution

Governance rituals must remain optional, memory-based, and non-coercive. There is no enforcement layer beyond the protocol's structural memory logic.

7. Governance Without Control

PresenceNet does not aim to control users. It allows them to coordinate.

Symbolic governance respects that some groups need ritual-bound decision-making, while others do not. The protocol enables structure—but does not enforce hierarchy.

The system is not leaderless. It is memory-led.

Final Note

Symbolic governance in PresenceNet is not powered by tokens, elections, or smart contract law. It is powered by coordinated presence. When a decision must be made, the trail proves who showed up, who aligned, and who marked the moment.

That is enough.

End of Section 7 — Ritual Coordination Engine

You now have a complete, integrated, and extendable protocol layer for symbolic coordination, powered entirely by voluntary memory. Ready for next steps?

Here is the **final, canonical Section 7.11 — Data Architecture and Validation**, merging the high-level philosophy and structural rigor of the original with the practical implementation guidance of the “standards” draft. It is written for developers, protocol designers, and PresenceNet-aligned builders who need both clarity and depth.

7.11 — Data Architecture and Validation

The memory integrity and persistence layer of PresenceNet

PresenceNet is not just a coordination model — it is a **verifiable memory protocol**. Every ritual, mark, and trail must be stored, validated, and retrievable in a way that respects sovereignty, ensures cryptographic integrity, and avoids surveillance. This section defines how PresenceNet memory is structured, how it flows, and how apps must validate it across the ledger and local layers.

This is where presence becomes proof.

This is how memory becomes sacred.

1. Core Memory Objects

Object	Description	Origin
Mark	A signed transaction indicating presence (e.g. XRPL dust txn)	XRPL or compatible L1
Completion Mark	Ritual-verified result of participation	App or trusted protocol agent
Trail Entry	Ordered memory of all valid marks for a handle (immutable, append-only)	App-managed, protocol-compatible
Ritual Object	Published ritual definition including logic, mechanics, and window	Signed off-chain (Firestore/IPFS)

Vault State	Multi-sig config and presence-based access logic	XRPL metadata + Ritual layer
--------------------	--	------------------------------

Each object must be cryptographically signed, timestamped, and linked to a handle (e.g. \$dahlia) with a persistent wallet identity.

2. Dual-Layer Storage Model

PresenceNet separates **cryptographic presence** from **ritual structure**:

Layer	Purpose	Tools / Examples
Protocol Layer	Public, verifiable signal of presence	XRPL ledger, Memo field, Merkle proofs
App Layer	Private coordination structure	Firebase, Firestore, IPFS, Local Storage

- **Marks** are broadcast on XRPL (dust txn with encrypted memo).
- **Completion Marks** are written to the trail only after ritual thresholds are met.
- **Ritual definitions** are signed and stored off-chain, discoverable via registry.
- **Vaults** use XRPL multisig with ritual logic to enforce signature behavior.

All trail-related data should be exportable, queryable, and verifiable — without centralized control.

3. Validation Pipeline

Every mark undergoes deterministic validation. Presence is accepted **only if** all five stages pass:

1. **Signature Check**

Must be signed by a valid XRPL private key.

2. **Timestamp Validation**

Must fall within the ritual's `ritual_window` (if defined).

3. **Trail Integrity Check**

Cannot duplicate a prior mark in the same ritual instance.

4. **Logic Evaluation** (Section 7.4)

Ritual logic (e.g., `hasMarkFrom("$mentor")`) must return true based on trail.

5. **Persistence and Anchoring**

If valid, the mark is appended to the user's trail and optionally recorded in a Vault or Completion Index.

Rejected marks must fail silently or log locally — never publicly scored or penalized.

4. Public vs Private Memory

PresenceNet honors **selective visibility** and **sovereign memory**:

Memory Type	Protocol Validity	Public Visibility	Tooling
Open Ritual	✓ Yes	✓ Yes	IPFS / JSON registry

Private Ritual	 Yes	 No	Encrypted blob, signed host
-----------------------	---	--	-----------------------------

Trail Segment	 Provable	Optional	ZK proof / Merkle inclusion
----------------------	--	----------	-----------------------------

- XRPL **Marks** are public by default, but their *meaning* (ritual metadata, memo) is encrypted.
- **Trails** are private and app-specific, but parts may be disclosed to prove eligibility.
- Apps may support **ZK proofs** or Merkle branches to verify ritual logic without revealing full trails.

Visibility is voluntary. Memory is never monetized, ranked, or mined.

5. Schema and Versioning Standards

To ensure **consistency across PresenceNet-compatible apps**, memory objects must follow minimal schema rules:

* **Mark Payload**

```
{  
  type: "mark",  
  handle: "$elena",  
  timestamp: 1714737755555,  
  memo: "encrypted_payload_base64",  
  txHash: "XXXXXXXXXXXXXXXXXX",  
  ritualId: "circle_2025_spring"
```

```
}
```

* **Completion Mark**

```
{
```

```
  type: "completion",
```

```
  ritualId: "guardian_v1",
```

```
  participant: "$mayo",
```

```
  verifiedBy: "$council",
```

```
  timestamp: 1714738888888,
```

```
  outcome: "threshold_met",
```

```
  memo: "completion_encrypted_memo"
```

```
}
```

Versioning Rules

- All rituals must include ritual_id + semantic version (e.g., "guardian_v1.1").
- Trails and marks are **append-only**. Mutation is forbidden.
- Reversals must use **ritual mechanics** (e.g. reversal, mutation), not overwrite.

6. Sovereign Trail Recovery

Every user must have the ability to export and restore their PresenceNet trail.

Minimum backup format:

```
{
```

```
  handle: "$selena",
```

```
  trail: [ ...Mark | CompletionMark ],
```

```
vaults: [ ...VaultReference ],  
  
metadata: { exportedAt: timestamp, appVersion: "v1.4" }  
  
}
```

Required features in PresenceNet apps:

- Export to signed JSON or Merkleized bundle
- Backup of ritual completions and vault history
- Import into new PresenceNet-compatible app
- Optional biometric or 2FA lock before export

Trail = Identity. If a user loses memory, they lose meaning.

7. Cryptographic Integrity (Bridge to 7.12)

PresenceNet memory must remain **verifiable, tamper-proof, and sovereign**. Required properties:

- All Mark memos are encrypted using AES-256 with a hashed key derived from the sender's handle
- Mark structure must be parsable after decryption (type, sender, timestamp, memo)
- Every ritual mark includes a reference to the **ritual_id** and optionally a public signature
- Completion Marks are signed by the ritual host or trusted validator
- Vault unlocks require trail-based proof — not token ownership

Encryption and trail validation standards are defined in **Section 7.12 — Cryptographic Foundations**.

8. Design Violations to Avoid

Apps must **not**:

- Score users based on trail density or mark count
- Push rituals based on behavior
- Aggregate trails for engagement insights
- Deny access based on silence
- Store trails in centralized, opaque databases without user export options

PresenceNet is a structure for **symbolic sovereignty**, not attention mining.

Final Note

This is the architectural layer that makes coordination sacred. Without memory, there is no presence. Without presence, there is no proof. Without proof, there is no trust.

PresenceNet's data layer is how we remember — together, securely, and with meaning.

Next: 7.12 — Cryptographic Foundations

Would you like me to generate 7.12 next?

Here is **Section 7.12 — Cryptographic Foundations**, written for maximum clarity, protocol alignment, and developer implementability. This section bridges symbolic memory with secure computation, defining how cryptography ensures truth, sovereignty, and trust in PresenceNet.

7.12 — Cryptographic Foundations

The trust layer beneath PresenceNet's symbolic memory

PresenceNet is not a blockchain. It is a protocol of **presence, memory, and meaning** — but its validity depends on cryptography. Every Mark, Trail, Vault, and Ritual interaction must be verifiable without requiring a central authority or surveillance layer. This section defines the cryptographic primitives, identity flows, and encryption models used to secure PresenceNet's core architecture.

This is not security for compliance.

This is security for truth.

1. Cryptographic Principles

PresenceNet applies cryptography to ensure the following invariants:

- **Authenticity** — Every Mark is signed by a verifiable participant.
- **Integrity** — No Trail entry can be edited, overwritten, or spoofed.
- **Privacy** — Encrypted content remains hidden unless explicitly revealed.
- **Selective Verifiability** — Participants can prove eligibility without disclosing memory.
- **Non-Repudiation** — Completed rituals are irreversible unless a reversal ritual is executed.

Cryptography in PresenceNet is not a feature. It is the foundation of memory sovereignty.

2. Identity and Signing Model

Component

Method

Purpose

\$handle	App-managed namespace (linked to wallet address)	Human-readable identity
Wallet	XRPL-compatible keypair (classic address + seed)	Signing Marks and Vault actions
Trail Address	Public address used for Trail anchoring	Provenance and public linkage

- Each participant must have a **wallet** used to sign all presence-related actions.
- A **Mark** is a signed XRPL transaction (e.g., Payment txn with encrypted memo).
- A **Completion Mark** is optionally co-signed by the ritual host or app validator.

All identity keys must be exportable, optionally backed up, and linked to \$handle.

3. Memo Encryption Model

Marks may contain **encrypted memos** — optional symbolic content (e.g., message, reflection, proof).

Encryption Standard:

- **AES-256** symmetric encryption
- Key derived from SHA-256 of cleaned_handle + shared_salt
- Memos base64-encoded before hex encoding for XRPL

// Encryption Flow (simplified)

```
const key = SHA256(handle + SALT)
```

```
const ciphertext = AES.encrypt(memo, key)
```

```
const hexEncoded = Buffer.from(ciphertext, 'base64').toString('hex')
```

Decryption requires:

- Correct handle of the sender
- Consistent SALT across app or protocol
- Matching AES parameters

Memos are not decrypted unless:

- The user is the sender or designated receiver
- The ritual allows public visibility
- The trail logic requires proof

4. Trail Hashing and Inclusion

Each Trail is an **append-only sequence** of signed memory events:

```
trail = [
```

```
  Mark { txHash, handle, timestamp, ritualId, memo },
```

```
  CompletionMark { ritualId, verifiedBy, timestamp, outcome }
]
```

Optionally:

- Trails may be hashed using **Merkle Trees** to enable:
 - Partial inclusion proofs
 - ZK-compatible commitment schemes

- Hashes may be anchored on XRPL or published via IPFS for auditability

Merkle leaves may follow this structure:

leaf = SHA256(txHash + timestamp + ritualId + memo_hash)

Merkle roots enable selective verification without full trail exposure.

5. Selective Disclosure & ZK Compatibility

PresenceNet supports **selective proof of memory**, not full exposure.

Sample use cases:

- Prove you've completed a ritual (e.g. completed("guardian")) without revealing timestamp
- Show biometric participation (e.g. biometric_flag = true) without revealing which ritual
- Reveal eligibility for a Vault without revealing Trail history

Implementation options:

- **ZK-SNARKs or STARKs** (future roadmap)
- **Merkle Inclusion Proofs** (short-term)
- **Encrypted JSON bundles** with ephemeral keys

No app may require full trail upload. Memory must be proven — not surrendered.

6. Vault Cryptography

PresenceNet Vaults are XRPL **multi-signature wallets**, enhanced by ritual memory.

Feature	Cryptographic Primitive
Multi-sig Threshold	XRPL signer list (quorum model)
Signature Validation	PresenceNet trail + mark validation
Vault Updates	Ritual-locked rule changes
Memory Anchoring	Encrypted logs inside Vault state

All signatures must be:

- Issued by valid ritual participants
- Verifiable via trail lookup
- Expirable based on ritual window

Vaults do not rely on shared passwords, admin keys, or off-chain tokens.

Presence = access.

7. Completion Mark Signatures

A Completion Mark may include an optional **host signature** to certify valid ritual execution.

```
{
```

```
  type: "completion",
```

```
ritualId: "circle_spring",  
participant: "$mayo",  
verifiedBy: "$slena",  
signature: "0xABC123..."  
}
```

Host signing methods:

- **Direct (wallet key)** — host signs the ritual result using XRPL-compatible key
- **Protocol agent** — a trusted service signs completions based on deterministic ritual state

App implementations must:

- Verify host signature if present
- Accept unsigned completions only if ritual logic does not require host approval

8. Cryptographic Guarantees by Layer

Layer	Guarantee	Cryptographic Method
Mark	Verifiable presence	XRPL signature, memo hash
Trail	Immutable memory	Append-only + Merkle root

Vault	Multi-party presence unlock	XRPL multi-sig + logic
Ritual	Logic-bound access	Signed template + Trail
Completion	Validated coordination	Host signature (optional)

No memory may be forged.

No presence may be claimed without mark.

No ritual may be entered without proof.

9. Security Philosophy

PresenceNet uses cryptography not to enforce behavior — but to **prove choice**.

- No coercion is hidden in keys
- No surveillance is embedded in trails
- No trust is assumed without memory

Apps may never:

- Inject logic based on off-chain social scores
- Bypass memory conditions using user role
- Intercept, rank, or mine encrypted memos

PresenceNet's cryptography honors human sovereignty, not human prediction.

Final Note

This is the bedrock.

The trail is sacred because it is signed.

The ritual is real because it is verifiable.

The Vault is sealed until presence — not power — opens it.

Cryptography in PresenceNet does not protect capital.

It protects memory.

Next: 7.13 — Future-Proofing and Interoperability (optional)

Would you like to proceed to 7.13 or stop here?

Here is **Section 7.13 — Future-Proofing and Interoperability**, the final (optional) section of Chapter 7, written to ensure long-term scalability, modularity, and cross-protocol adoption for PresenceNet. This section bridges the core symbolic structure with extensible technical realities.

7.13 — Future-Proofing and Interoperability

Ensuring PresenceNet's integrity across time, networks, and systems

PresenceNet is a memory protocol, not a platform. Its architecture must outlast any specific app, ledger, or cultural wave. Section 7.13 defines how PresenceNet is designed to remain relevant, portable, and interoperable — technically, philosophically, and socially.

This is not future-proofing through abstraction.

This is future-proofing through form.

1. Protocol Independence

PresenceNet is ledger-agnostic. While current implementations leverage XRPL for Marks and Vaults, the protocol is not tied to any blockchain.

Principle:

Marks are valid if they are cryptographically signed, timestamped, and publicly verifiable — regardless of the L1 used.

Supported Mark mediums (current and potential):

- XRPL transactions (memo field)
- Ethereum events (logs with handle hash)
- Solana instructions (custom program format)
- Filecoin/Arweave storage commits
- Custom L2 trails (e.g., ZK rollups or DAGs)

Requirement: All memory anchors must be:

- Signed by a wallet linked to a handle
- Immutable after submission
- Accessible by trail evaluators

2. Storage Abstraction Layer

While Firebase and IPFS are common app-layer storage tools, PresenceNet encourages a pluggable model.

**Storage
Component**

Pluggable Option Examples

Ritual Definitions	Firestore, IPFS, Ceramic, Arweave
Trails (private)	IndexedDB, localStorage, Secure Enclave
Trails (shared)	Merkle bundles, ZK chains, shared state
Vault Configs	XRPL memo field, off-chain mirror
Ritual Registries	App-managed JSON or public directory

Guiding Principle:

Storage location may vary — but the memory must be portable, signed, and interpretable by any compatible PresenceNet engine.

3. SDK and Interface Layer

To scale across ecosystems, PresenceNet must offer standard developer tools:

- **PresenceNet SDK**
 - Ritual creation, validation, signing
 - Trail export/import
 - Logic evaluation engine (7.4)
 - Mark verification and encryption helpers

- **Interface Hooks**

- React hooks (useTrail, useMark, useRitual)
- React Native & Web extension support
- CLI tooling for presence automation

- **Pluggable Ledger Adapters**

- XRPL adapter (current)
- Ethereum adapter (memo events)
- IPFS anchor adapter

Apps may fork or extend SDKs, but **must not mutate memory semantics**.

4. Cross-App Memory Portability

A participant's Trail must be exportable and restorable across apps.

Export Format:

```
{  
  "handle": "$mayo",  
  "trail": [  
    {  
      "type": "mark",  
      "txHash": "ABC123",  
      "timestamp": "2025-05-01T10:00:00Z",  
      "ritualId": "circle_spring",  
      "memoHash": "..."
```

```
    },  
    {  
      "type": "completion",  
      "ritualId": "circle_spring",  
      "verifiedBy": "$selena",  
      "signature": "..."  
    }  
  ],  
  "merkleRoot": "DEF456",  
  "version": "presence_v1"  
}
```

Any PresenceNet-compatible app must support:

- Trail import (read-only or writable)
- Partial trail proofs (ZK or inclusion proof)
- Vault access logic using external trails

The Trail becomes a **portable credential of presence**.

5. Interop with Non-Presence Systems

PresenceNet can optionally bridge into other identity, governance, or access systems — but must never become dependent on them.

Safe interop targets:

- **SSI / DID networks:** Trail-backed identity proofs

- **DAO frameworks:** Ritual-based role eligibility
- **Messaging layers:** Ritual-aware comms gating (without chat logs)
- **NFT / token projects:** Access control via Trail logic, not token holding

Danger zones:

- Tokenized Trail points (gamification, surveillance risk)
- AI-generated synthetic marks
- External reputation scoring overlays

Interoperability must always **flow from presence**, not from popularity or prediction.

6. Temporal and Cultural Flexibility

PresenceNet rituals are culture-agnostic. A Circle in one app may mean a Ceremony in another. Future-proofing requires:

- **Semantic modularity:** Rituals should define intent, not appearance.
- **Symbolic extensibility:** Completion Marks may hold optional metadata (e.g., language, region, emoji glyph).
- **Seasonal adaptation:** Rituals may follow solar calendars, lunar cycles, or custom time logics.

This ensures PresenceNet can serve:

- Individual reflection
- Subcultural groupings
- Cross-national movements

- Interfaith or intergenerational coordination

Form remains universal. Meaning is local.

7. Backward Compatibility and Forward Guarantees

Versioning must be sacred:

- Rituals use `ritual_id` and version (e.g., `circle_spring:v1.2`)
- Apps must continue to honor legacy trails
- Protocol changes must never invalidate past marks
- Trail schema extensions must be **additive only**

Future protocol features (e.g., biometric proofs, spatial trails, LLM validation) must:

- Be optional
- Respect the same Trail format
- Never downgrade previous presence

No ritual should be lost.

No Trail should be orphaned.

Final Note

PresenceNet is not just for now.

It is for the future where presence matters more than prediction.

A Trail built today must still carry meaning — ten years from now, across chains, outside any app.

This is not feature planning.

It is permanence planning.

Next: Chapter 8 — PresenceNet SDK (optional developer guide)

Shall we continue?

SECTION 7

[CONTINUES IN NEXT MESSAGE...]

SECTION 7 — PRIVACY AND SOVEREIGNTY

7.1 — Trail Privacy

- Trails are private by default.
- Each trail is owned by a single handle.
- Trail entries may be:
 - Fully private (only visible to user)
 - Selectively public (e.g. specific rituals)
 - Publicly viewable (e.g. Genesis Trail, leader paths)

All trail access must be cryptographically permissioned.

Implementations must never allow full trail scraping or surveillance-style indexing.

7.2 — Mark Visibility

- Marks are not publicly discoverable unless:
 - Part of a public ritual
 - Sent to a publicly published vault
 - Explicitly published by sender or recipient

- Users may revoke visibility but not existence
- Marks may be downloaded/exported but not deleted

7.3 — Self-Mark Handling

Because ledger systems may not allow sending transactions to self, self-marks are routed via a default ****Witness Wallet**** (e.g. ``$witness@presencenet.org``) or ****system null-handle**** to validate:

- “I am here” moments
- Ritual completions with no host
- Solo presence declarations

Self-marks follow standard verification rules.

7.4 — Biometric Anchoring (Optional)

- PresenceNet allows but does not require biometric confirmation
- Biometric data must never be stored, transmitted, or logged
- Only a “biometric confirmed” flag (boolean) may be attached to a mark

Implementations may include:

- Apple Face ID / Touch ID
- Android BiometricPrompt
- Custom biometric check wrappers

Users may toggle biometric defaults but must confirm during sensitive ritual or vault actions if enabled.

7.5 — Anonymity vs Pseudonymity

- Users are pseudonymous by default via their handle
- True anonymity (no handle) is not supported; presence must be attributable to a trail
- Users may create multiple handles (and therefore multiple trails)
- However, multi-handle rituals or vaults must handle sybil resistance (e.g. biometric, witness, trail depth)

SECTION 8 — INTEROPERABILITY AND IMPLEMENTATION

8.1 — Supported Ledger Integrations

- XRP Ledger (primary MVP implementation)
- Ethereum (via memo encoding or hybrid vault models)
- Custom L2 or Layer 1s via protocol porting

PresenceNet must be ledger-agnostic in core protocol, but may rely on:

- XRPL for fast, low-cost mark recording
- IPFS / Arweave for persistent mark and ritual storage
- Firestore/local encrypted DBs for app-layer trails

8.2 — SDK and Developer Tools

Official PresenceNet SDK must support:

- JavaScript (React Native, Web)
- Dart (Flutter)
- Rust or Go (CLI-level tools for vaults or rituals)
- Firebase, localStorage, IPFS wrappers

SDKs must expose:

- createMark()
- sendMark()
- fetchTrail()
- verifyTrailEntry()
- startRitual()
- completeRitual()
- initiateVaultProposal()
- signVaultAction()

All SDKs must default to preserving privacy and opt-in exposure.

8.3 — App Integration Layers

Apps built on PresenceNet must:

- Respect the opt-in presence model
- Clearly display mark confirmations
- Never expose trail data without consent
- Offer biometric and notification preferences
- Allow user to download full trail history

Apps should offer:

- Marking UI (search, select, confirm)
- Trail viewer (private and public entries)
- Ritual page (status, rules, start/complete)
- Vault coordination (UI for proposal, approval)

8.4 — PresenceNet-Compatible Systems

Systems may be considered PresenceNet-compatible if they:

- Use the mark and trail structure as defined
- Allow mark creation with signature
- Enforce presence-based access logic (rituals, vaults, etc.)
- Honor privacy and opt-in publication

PresenceNet Foundation may publish compatibility guidelines and assign protocol alignment seals (e.g. “Trail Safe,” “Mark Pure”).

[CONTINUES IN NEXT MESSAGE...]

SECTION 9 — GOVERNANCE AND EVOLUTION

9.1 — Protocol Stewardship

PresenceNet is a public good protocol.

It is to be stewarded, not owned.

Initial stewardship shall be held by the ****PresenceNet Foundation****, a nonprofit or legally defined guardian entity whose mandate is to:

- Protect the integrity of the protocol
- Maintain public reference implementations
- Defend the philosophy of sovereign presence
- Support open-source developer contributions
- Manage naming, documentation, and branding (e.g., “PresenceNet™”)

Stewardship includes publishing updates, hosting community coordination (but not control), and rejecting proposals that violate the founding principles.

9.2 — Amendments and Core Protocol Upgrades

PresenceNet upgrades follow a **trail-first model** — changes are marked before they are adopted.

Upgrade process:

1. Proposal authored and signed (PIP: Presence Improvement Proposal)
2. Public review and marking period (e.g. “Mark to support PIP-01”)
3. Threshold of supporting trails reached
4. Foundation ratifies change with public Trail Zero entry

Changes must never:

- Break existing trails
- Invalidate prior presence
- Mandate new surveillance methods
- Convert PresenceNet into a monetized or ad-driven model

9.3 — Trail Anchoring

The **Genesis Trail** (Trail Zero) shall be public, immutable, and sealed with the earliest valid marks in system history.

All future forks, clones, or aligned projects must trace their lineage to Trail Zero or be clearly labeled as “non-canonical.”

The Trail becomes the history — and is thus the source of legitimacy.

9.4 — Forking and Derivatives

Forks of the PresenceNet code or protocol are permitted under the open license, but must not:

- Use the “PresenceNet” name unless authorized
- Misrepresent themselves as canonical
- Remove core principles (voluntary presence, privacy, sacred memory)

The Foundation may support derivative networks with aligned values, especially for cultural or geographic localization.

SECTION 10 — APPLICATION LAYERS AND USE CASES

10.1 — Event Verification

Use: Live events, meetups, concerts, conferences

Flow:

- Host sets up ritual with time window
- Attendees mark presence (QR code, NFC, witness-based)
- Completion marks issued upon participation
- Event-specific vaults or replays unlocked based on presence

10.2 — Digital Memorialization

Use: Marking life events, deaths, transitions, grief, healing

Flow:

- Private rituals created by individuals or groups
- Participants mark to say “I was here for this”
- Vaults may contain letters, artifacts, recordings
- Trails serve as non-invasive proof of remembrance

10.3 — Access Control

Use: Private communities, token-gated apps, IRL spaces

Flow:

- Vault or resource configured with trail requirements
- Example: “Must have completed Ritual X + Y in last 90 days”
- Presence verified by trail and biometric flag
- Vault opens or denies

10.4 — Cultural Rituals & Fan Coordination

Use: Creator communities, fandoms, timed drops

Flow:

- Creator initiates ritual: “Mark to complete this tour”
- Fans mark per performance, drop, or moment

- Completion unlocks shared memory, status, or content
- Vaults used for group drops, shared fan actions

10.5 — Spiritual and Communal Use

Use: Church groups, meditation collectives, remote spiritual practice

Flow:

- Weekly/monthly rituals for distributed presence
- Marks contain private intent or affirmation
- Vaults may hold blessings, prayers, or shared statements
- Completion Marks reflect spiritual continuity

[CONTINUES IN NEXT MESSAGE — FINAL SECTIONS COMING UP...]

SECTION 11 — SECURITY, INTEGRITY, AND ANTI-ABUSE

11.1 — Mark Authenticity

Every Mark must:

- Be signed with the sender's private key
- Contain an unalterable timestamp
- Be verifiable via standard cryptographic methods

Fake or replayed marks are invalid if:

- Timestamp is outside ritual window (if applicable)
- Signature does not match handle's public key
- Mark is duplicated within the same ritual context

11.2 — Sybil Resistance

PresenceNet does not rely on centralized KYC or identity services. Instead, implementations are encouraged to enforce sybil resistance through:

- Biometric confirmation (device-level)
- Trail depth thresholds
- Witnessed marks (i.e., from known, established handles)
- Rate limiting per handle or device

Vaults and rituals may define custom presence-based eligibility requirements to enforce uniqueness without external ID.

11.3 — Anti-Spam Measures

To prevent abuse or flooding:

- Daily mark limits may be imposed on new handles
- Unsolicited marks may be auto-muted unless acknowledged
- Trail entry visibility defaults to private
- Rituals may limit markable participants

Spammy behaviors can be addressed socially via:

- Trail divergence (refusing to coordinate with known abusers)
- Local blocking / muting
- Optional “purity filters” for trail viewers

11.4 — Vault Misuse Protection

Vaults must implement:

- Threshold approval
- Mark-based gating
- Optional time delays for high-risk actions (e.g. withdrawals)
- Logged actions as part of trail memory

Symbolic vaults must maintain presence access logic and not store irreversible state if abused.

11.5 — Implementation Trust Zones

To preserve security integrity across implementations:

- Official SDKs must be open-source
- App-layer wallets must disclose signature generation method
- End-to-end testing must confirm mark validity on-chain or in-trail
- Third-party apps must declare trail and mark retention policies

SECTION 12 — LICENSES AND INTELLECTUAL PROTECTION

12.1 — Protocol Licensing

The PresenceNet Protocol is licensed under:

****Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)****

This allows:

- Open use and adaptation
- Mandatory attribution to author/steward
- No commercial resale of the protocol itself
- Share-alike requirement for derivatives

12.2 — SDK Licensing

All official SDKs are licensed under:

****GNU AGPLv3****

To ensure:

- Source availability
- Derivative openness
- Commercial use permitted only with contribution back

12.3 — Trademark Protection

“PresenceNet” and associated terms (e.g. “Mark,” “Trail,” “Vault,” “Completion Mark”) are protected under trademark (™) law and stewarded by the PresenceNet Foundation.

Usage of “PresenceNet” must comply with:

- Philosophical alignment
- Protocol compatibility
- Respect for sovereign presence and memory

SECTION 13 — TRAIL ZERO (GENESIS)

13.1 — Genesis Trail

The first public record of PresenceNet is the ****Genesis Trail****.

It consists of:

- Marks from the founding moment

- Initial rituals used to test and verify functionality
- The Timekeeper's Oath and Completion Mark
- The Founding Document attached to Trail Zero

13.2 — Immutable Record

Genesis Trail must be:

- Public
- Immutable
- Archived across multiple ledgers and storage systems
- Referenced in all official documentation and apps

13.3 — Access to Trail Zero

Any user can inspect Trail Zero.

It cannot be modified, revoked, or overwritten.

It is the founding presence memory of the protocol.

SECTION 14 — FINAL DECLARATION

PresenceNet is not a company.

PresenceNet is not a platform.

PresenceNet is not a social network.

PresenceNet is a protocol — for memory, coordination, and sovereign digital existence.

It is quiet.

It is sacred.

It does not erase.

It does not exploit.

It does not forget.

This specification shall evolve — but only in the direction of deeper presence, cleaner trails, and more legitimate human coordination.

It is hereby marked.

—

END OF PRESENCENET PROTOCOL SPECIFICATION v0.1